

Pair Sum Problem:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter array elements:");
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter target value: ");
        int target = sc.nextInt();

        boolean found = false;

        for(int i = 0; i < arr.length; i++) {

            for(int j = i + 1; j < arr.length; j++) {
```

```

        if(arr[i] + arr[j] == target) {
            System.out.println(arr[i] + " + " + arr[j] + " = " + target);
            found = true;
        }
    }
}

if(!found) {
    System.out.println("No pair found");
}

sc.close();
}
}

```

Triplet Sum Problem:

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();
    }
}

```

```
int[] arr = new int[n];
```

```
System.out.println("Enter array elements:");
```

```
for(int i = 0; i < n; i++) {
```

```
    arr[i] = sc.nextInt();
```

```
}
```

```
System.out.print("Enter target value: ");
```

```
int target = sc.nextInt();
```

```
boolean found = false;
```

```
for(int i = 0; i < arr.length; i++) {
```

```
    for(int j = i + 1; j < arr.length; j++) {
```

```
        if(arr[i] + arr[j] == target) {
```

```
            System.out.println(arr[i] + " + " + arr[j] + " = " + target);
```

```
            found = true;
```

```
        }
```

```
    }
```

```
}
```

```
if(!found) {
```

```
    System.out.println("No pair found");
```

```
    }  
  
    sc.close();  
}  
}
```

Maximun Subarray Sum:

```
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter array size: ");  
        int n = sc.nextInt();  
  
        int[] arr = new int[n];  
  
        System.out.println("Enter array elements:");  
        for(int i = 0; i < n; i++) {  
            arr[i] = sc.nextInt();  
        }  
  
        int currentSum = arr[0];
```

```

int maxSum = arr[0];

for(int i = 1; i < n; i++) {

    currentSum = Math.max(arr[i], currentSum + arr[i]);

    maxSum = Math.max(maxSum, currentSum);
}

System.out.println("Maximum Subarray Sum = " + maxSum);

sc.close();
}
}

```

Move Zero to End:

```

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");

        int n = sc.nextInt();
    }
}

```

```
int[] arr = new int[n];
```

```
System.out.println("Enter array elements:");
```

```
for(int i = 0; i < n; i++) {  
    arr[i] = sc.nextInt();  
}
```

```
int index = 0;
```

```
// Move non-zero elements to front
```

```
for(int i = 0; i < n; i++) {  
    if(arr[i] != 0) {  
        arr[index] = arr[i];  
        index++;  
    }  
}
```

```
// Fill remaining positions with zero
```

```
while(index < n) {  
    arr[index] = 0;  
    index++;  
}
```

```
System.out.println("Array after moving zeros to end:");
```

```
for(int i = 0; i < n; i++) {
```

```
        System.out.print(arr[i] + " ");
    }

    sc.close();
}
}
```

Merge Two Sorted Arrays:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n1 = sc.nextInt();
        int[] arr1 = new int[n1];

        for(int i = 0; i < n1; i++) {
            arr1[i] = sc.nextInt();
        }

        int n2 = sc.nextInt();
        int[] arr2 = new int[n2];

        for(int i = 0; i < n2; i++) {
```

```
        arr2[i] = sc.nextInt();
    }

    int[] result = new int[n1 + n2];

    int i = 0, j = 0, k = 0;

    while(i < n1 && j < n2) {

        if(arr1[i] <= arr2[j]) {
            result[k++] = arr1[i++];
        } else {
            result[k++] = arr2[j++];
        }
    }

    while(i < n1) {
        result[k++] = arr1[i++];
    }

    while(j < n2) {
        result[k++] = arr2[j++];
    }

    for(int x : result) {
        System.out.print(x + " ");
    }
}
```



```
    }  
    }  
}
```

Equilibrium Index:

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter array size: ");
```

```
        int n = sc.nextInt();
```

```
        int[] arr = new int[n];
```

```
        System.out.println("Enter array elements:");
```

```
        for(int i = 0; i < n; i++) {
```

```
            arr[i] = sc.nextInt();
```

```
        }
```

```
        int totalSum = 0;
```

```
        for(int i = 0; i < n; i++) {
```

```
            totalSum += arr[i];
```

```
}
```

```
int leftSum = 0;
```

```
boolean found = false;
```

```
for(int i = 0; i < n; i++) {
```

```
    totalSum = totalSum - arr[i];
```

```
    if(leftSum == totalSum) {
```

```
        System.out.println("Equilibrium Index = " + i);
```

```
        found = true;
```

```
        break;
```

```
    }
```

```
    leftSum += arr[i];
```

```
}
```

```
if(!found) {
```

```
    System.out.println("No Equilibrium Index Found");
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```

Problem Statement: Reverse a String

Given a string **S**, write a program to reverse the string and print the reversed string.

Input Format

- A single line containing the string **S**.

Output Format

- Print the reversed string.

Constraints

- $1 \leq \text{Length of } S \leq 1000$
-

Sample Input 2

Java

Sample Output 2

avaJ

Explanation

The reverse of "Java" is "avaJ".

Sample Input 3

Programming

Sample Output 3

gnimmargorP

Java Program

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
String str = sc.nextLine();

String reverse = "";

for (int i = str.length() - 1; i >= 0; i--) {
    reverse += str.charAt(i);
}

System.out.println(reverse);
}
}
```

Problem Statement: Palindrome String

Given a string **S**, check whether it is a palindrome or not.

A string is called a **palindrome** if it reads the same forwards and backwards.

If the string is a palindrome, print **"Palindrome"**. Otherwise, print **"Not Palindrome"**.

Input Format

- A single line containing the string **S**.

Output Format

- Print **"Palindrome"** if the string is a palindrome.
 - Otherwise, print **"Not Palindrome"**.
-

Constraints

- $1 \leq \text{Length of } S \leq 1000$
 - String contains only alphabets.
-

Sample Input 1

madam

Sample Output 1

Palindrome

Explanation

Original String = "madam"

Reversed String = "madam"

Both are equal, so it is a palindrome.

Sample Input 2

hello

Sample Output 2

Not Palindrome

Explanation

Original String = "hello"

Reversed String = "olleh"

Both are not equal, so it is not a palindrome.

Sample Input 3

racecar

Sample Output 3

Palindrome

Java Program

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
String str = sc.nextLine();
String reverse = "";

for (int i = str.length() - 1; i >= 0; i--) {
    reverse += str.charAt(i);
}

if (str.equals(reverse)) {
    System.out.println("Palindrome");
} else {
    System.out.println("Not Palindrome");
}
}
```

Problem Statement: Count Vowels and Consonants

Given a string **S**, write a program to count the number of vowels and consonants present in the string.

Vowels

The vowels are:

a, e, i, o, u

A, E, I, O, U

All other alphabets are considered consonants.

Input Format

- A single line containing the string **S**.

Output Format

Print the count of vowels and consonants in the following format:

Vowels: X

Consonants: Y

Where:

- X = Number of vowels
 - Y = Number of consonants
-

Constraints

- $1 \leq \text{Length of } S \leq 1000$
 - String contains only English alphabets.
-

Sample Input 1

hello

Sample Output 1

Vowels: 2

Consonants: 3

Explanation

String = "hello"

Vowels = e, o $\rightarrow$ 2

Consonants = h, l, l $\rightarrow$ 3

Sample Input 2

Programming

Sample Output 2

Vowels: 3

Consonants: 8

Explanation

Vowels = o, a, i $\rightarrow$ 3

Consonants = P, r, g, r, m, m, n, g $\rightarrow$ 8

Sample Input 3

AEIOU

Sample Output 3

Vowels: 5

Consonants: 0

Java Program

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        int vowels = 0;
        int consonants = 0;

        for (int i = 0; i < str.length(); i++) {

            char ch = Character.toLowerCase(str.charAt(i));

            if (ch == 'a' || ch == 'e' || ch == 'i' ||
                ch == 'o' || ch == 'u') {
                vowels++;
            } else {
                consonants++;
            }
        }

        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
    }
}
```